

Redes y Programación de Sockets

Antonio Espín Herranz

Contenidos

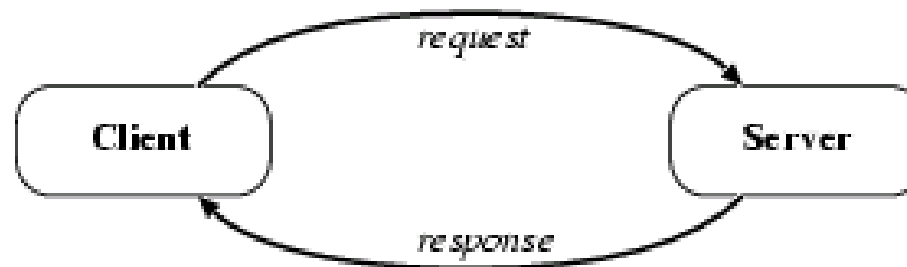
- Fundamentos de sockets
- Multiplexación de E/S
- Servidores concurrentes
- Protocolos y serialización

Fundamentos de sockets

- Sockets TCP y UDP — flujo vs datagramas.
- bind, listen, accept — servidor básico.
- connect, send, recv — cliente básico.

Socket

- Un socket se podría equiparar a un enchufe de la red eléctrica. Es un extremo de comunicación entre dos procesos.
- Los vamos a utilizar para comunicar procesos que pueden estar ejecutándose en la misma máquina localhost o en máquinas distintas (remotas).
- Necesitamos dos parámetros para conectar la dirección IP y el número de puerto.
- Tener en cuenta que los primeros 1024 número de puerto están reservados. Por ejemplo:
25 → smtp, 80 → http, 20, 21 → FTP.
- Los Sockets nos permiten trabajar con una arquitectura Cliente / Servidor.



Tipos de Sockets

- **TCP (SOCK_STREAM)**

- TCP es un protocolo *orientado a conexión*.
- Características:
 - Garantiza la entrega de los datos.
 - Garantiza el orden de recepción.
 - Retransmite paquetes perdidos.
 - Controla errores y congestión.
- Flujo de datos de dos vías, confiable, y sin duplicados sin límites de grabación

- **UDP (SOCK_DGRAM)**

- UDP es un protocolo *orientado a datagramas*.
- Características:
 - No garantiza entrega.
 - No garantiza orden.
 - No retransmite.
 - Menor latencia.
 - Menor sobrecarga.
- Un socket de datagrama podría recibir mensajes en diferente orden de la secuencia de la cual los mensajes fueron enviados

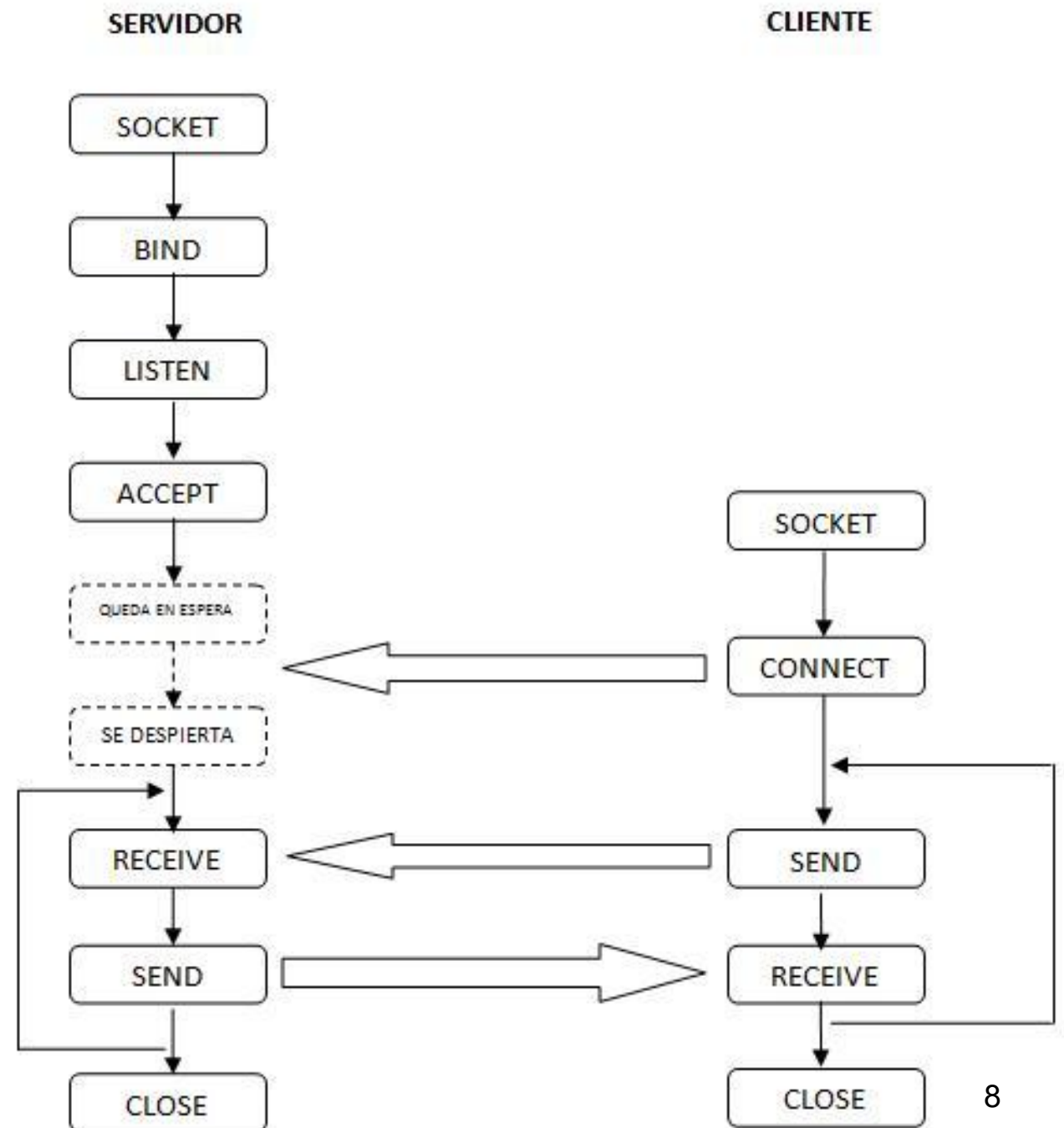
Uso típico de los Sockets

- **TCP (SOCK_STREAM)**
 - Transmission Control Protocol
 - **Usos:**
 - HTTP
 - HTTPS
 - SSH
 - Bases de datos
 - APIs REST
- **UDP (SOCK_DGRAM)**
 - User Datagram Protocol
 - **Usos:**
 - DNS
 - Streaming
 - VoIP
 - Videojuegos
 - Telemetría

Tipos de Dominios

- El dominio de comunicación nos dice como se va a realizar la comunicación de los procesos que se van a intercomunicar, o, mejor dicho, en que ambiente.
- Si los procesos se comunicarán bajo la forma de un único sistema (tipo root de Unix), el dominio de comunicación será **AF_UNIX**, si los procesos se comunicarán como sistemas independientes y estos se hallan unidos mediante una red TCP/IP, el dominio de comunicación será **AF_INET**.
- En nuestro caso vamos a utilizar el dominio (también llamado familia de protocolos) **AF_INET**.

Esquema Cliente / Servidor



Cliente / Servidor

- **Pasos que realiza el Servidor:**

- Crear el **Socket**.
- Configurar los datos de la dirección del Servidor (IP, puerto, etc).
- Ligar el socket con la dirección. (**Bind**).
- Poner el Socket del Servidor a la escucha para recibir peticiones por parte del Cliente. (**Listen**).
- Aceptar peticiones de los clientes (**Accept**).
- Una vez que hemos conectado con el cliente creamos un proceso hijo con **fork** y le podemos enviar datos mediante **Send**

- **Pasos que realiza el Cliente**

- Puede recuperar la información del host con el quiere conectar a partir de la información de la función **gethostbyname()**.
 - **gethostbyname("localhost")**
- Crear un **Socket** para comunicarse.
- Crear la dirección destino.
- Realizar **connect** para conectar con el Servidor.
- Recibir datos del Servidor mediante **recv**.

Cliente / Servidor

- La puesta en marcha se realiza de la siguiente manera:
 - Lanzar el Servidor.
 - Después lanzar el cliente indicando en este caso localhost (se manda como parámetro de main()).
 - También se puede probar un cliente, mediante el comando **Telnet**. (desde una consola de MSDOS)
 - Telnet localhost 3490 (Tenemos que indicar el puerto en el que hayamos arrancado el Servidor)

Estructuras utilizadas

- Ficheros de cabecera y las estructuras:
 - struct in_addr → #include <netinet/in.h>
 - struct hostent → #include <netdb.h>
 - struct sockaddr_in → #include <netinet/in.h>

Estructuras I

// Esta estructura representa una dirección IPv4

```
struct in_addr {  
    unsigned long s_addr;  
};
```

- El campo s_addr almacena una dirección IP en formato binario, no en texto.
- La conversión de texto a binario se realiza mediante una función:
 - struct in_addr addr;
 - **inet_pton**("127.0.0.1", &addr);
- O de esta otra forma:
 - **inet_pton**(AF_INET, "127.0.0.1", &addr);

Estructuras II

```
struct sockaddr_in {  
    short int sin_family;  
    unsigned short int sin_port;  
    struct in_addr sin_addr;  
};
```

```
struct sockaddr_in server;  
  
server.sin_family = AF_INET;  
  
server.sin_port = htons(8080);  
  
inet_pton(AF_INET, "127.0.0.1",  
          &server.sin_addr);
```

Estructuras III

// Esta estructura representa un HOST.

```
struct hostent {  
    char *h_name;           /* El nombre oficial del nodo. */  
    char **h_aliases;       /* Lista de Alias. */  
    int h_addrtype;         /* Tipo de dirección del nodo. */  
    int h_length;           /* Longitud de la dirección. */  
    char **h_addr_list;     /* Lista de direcciones del nombre del servidor. */  
};
```

- En versiones más antiguas se ponía:
 - `#define h_addr h_addr_list[0] /* Dir., para la compatibilidad con anteriores. */`
- La podemos utilizar para conocer los datos de un host a partir de su nombre:

```
struct hostent *he;  
he = gethostbyname("localhost");
```

El objetivo de la estructura
es almacenar toda la información
obtenida al resolver un nombre
como:

- localhost
- www.google.com
- www.microsoft.com

Campos de struct hostent

- **h_name**: Nombre oficial del host.
 - `printf("%s\n", he->h_name);`
- **h_aliases**: Lista de nombres alternativos:

```
char **alias = he->h_aliases;  
while (*alias) {  
    printf("%s\n", *alias);  
    alias++;  
}
```

Campos de struct hostent II

- **h_addrtype**: El tipo de la dirección devuelta.
 - Normalmente será AF_INET
 - `printf("%d\n", he->h_addrtype);`
- **h_length**: Tamaño de una dirección. En IPv4 suele ser 4 (los 32 bits que ocupa).
- **h_addr_list**: Contiene una lista de direcciones IP asociadas al host. Muchas veces un servidor tiene varias IPs:

socket()

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int socket(int domain,int type,int protocol);
```

- **domain.** Se podrá establecer como AF_INET (para usar los protocolos ARPA de Internet), o como AF_UNIX (si se desea crear sockets para la comunicación interna del sistema).
- **type.** Aquí se debe especificar la clase de socket que queremos usar (de Flujos o de Datagramas). Las variables que deben aparecer son SOCK_STREAM o SOCK_DGRAM según queramos usar sockets de Flujo o de Datagramas, respectivamente.
- **protocol.** Aquí, simplemente se puede establecer el protocolo a 0.

bind()

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int bind(int fd, struct sockaddr *my_addr,int addrlen);
```

- **fd.** Es el descriptor de fichero socket devuelto por la llamada a socket().
- **my_addr.** es un puntero a una estructura sockaddr
- **addrlen.** contiene la longitud de la estructura sockaddr a la cuál apunta el puntero my_addr. Se debera establecer como sizeof(struct sockaddr).
- La llamada bind(): Su función esencial es asociar un socket con un puerto (de nuestra máquina). Análogamente socket(), devolverá -1 en caso de error.

connect()

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int connect(int fd, struct sockaddr *serv_addr, int addrlen);
```

- **fd.** Deberá configurarse como el fichero descriptor del socket, el cuál fue devuelto por la llamada a socket().
- **serv_addr.** Es un puntero a la estructura sockaddr la cuál contiene la dirección IP destino y el puerto.
- **addrlen.** Análogamente de lo que pasaba con bind(), este argumento debera establecerse como sizeof(struct sockaddr).
- La función connect() se usa para conectarse a un puerto definido en una dirección IP, la utiliza el cliente para conectar con el Servidor. Devolverá -1 si ocurre algún error.

listen()

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int listen(int fd,int backlog);
```

- **fd.** Es el fichero descriptor del socket, el cual fue devuelto por la llamada a socket().
- **backlog.** Es el número de conexiones permitidas.
- La función listen() se usa si se están esperando conexiones entrantes, lo cual significa, si se quiere, que alguien pueda conectarse a nuestra máquina. La ejecutará el Servidor.
- Después de listen() se ejecutará accept() para aceptar a un cliente.

accept()

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int accept(int fd, void *addr, int *addrlen);
```

- **fd.** Es el fichero descriptor del socket, que fue devuelto por la llamada a listen().
- **addr.** Es un puntero a una estructura sockaddr_in en la que se pueda determinar que nodo nos está contactando y desde que puerto.
- **addrlen.** Es la longitud de la estructura a la que apunta el argumento addr, por lo que conviene establecerlo como sizeof(struct sockaddr_in), antes de que su dirección sea pasada a accept().

send()

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int send(int fd, const void *msg, int len, int flags);
```

- **fd.** Es el fichero descriptor del socket, con el cual se desea enviar datos.
- **msg.** Es un puntero apuntando al dato que se quiere enviar.
- **len.** es la longitud del dato que se quiere enviar (en bytes).
- **flags.** deberá ser establecido a 0.

recv()

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
int recv(int fd, void *buf, int len, unsigned int flags);
```

- **fd.** Es el descriptor del socket por el cual se leerán datos.
- **buf.** Es el búfer en el cual se guardará la información a recibir.
- **len.** Es la longitud máxima que podrá tener el búffer.
- **flags.** Por ahora, se deberá establecer como 0.

close()

```
#include <unistd.h>  
close(fd);
```

- La función close() es usada para cerrar la conexión de nuestro descriptor de socket.
- Si llamamos a close() no se podrá escribir o leer usando ese socket, y si alguien trata de hacerlo recibirá un mensaje de error.

Multiplexación de E/S

- **select** – monitorización de múltiples sockets
- **poll** – alternativa escalable
- **epoll** – alto rendimiento para Linux

Introducción

- Cuando un servidor trabaja con un solo cliente, la lógica es:
 - `accept()`
 - `recv()`
 - `send()`
 - `close()`
- Si tenemos varios clientes simultáneamente surgen problemas.

Introducción

- Si hacemos `recv()` en el primer cliente, la llamada queda bloqueada esperando datos del cliente.
- El resto de clientes, pueden estar:
 - el cliente 2 puede estar enviando datos,
 - el cliente 3 puede haber cerrado la conexión,
 - el cliente 4 puede estar esperando una respuesta.
- El servidor no podría atenderlos porque está bloqueado.

Introducción

- La multiplexación permite preguntar al servidor por todos los sockets en vez de bloquearse en uno solo.
- El kernel monitoriza múltiples descriptores y despierta al proceso únicamente cuando alguno de ellos:
 - recibe datos,
 - puede enviar datos,
 - se desconecta,
 - produce un error.

select

- Monitorización de **múltiples sockets**.
 - **select()** fue la primera solución estandarizada de Unix para vigilar varios descriptores simultáneamente.
- El programa construye una lista de sockets:
 - fd_set readfds
- Pregunta al kernel con select(...) que sockets están listos para leer.

select

- Como resultado de ejemplo, puede dar:
 - Socket 3 -> sin datos
 - Socket 4 -> sin datos
 - Socket 5 -> tiene datos
 - Socket 6 -> sin datos
- El servidor en este caso procesa únicamente el socket 5.

select

- **Estructura básica:**

```
fd_set readfds;  
FD_ZERO(&readfds);  
FD_SET(sockfd1, &readfds);  
FD_SET(sockfd2, &readfds);  
FD_SET(sockfd3, &readfds);  
select(maxfd + 1, &readfds, NULL, NULL, NULL);
```

- Después de la llamada permite saber si un socket tiene actividad:
FD_ISSET(sockfd2, &readfds)

LIMITACIONES:

Normalmente **FD_SIZE** es **1024**

Hay limitación en el número de sockets.

La escalabilidad es pobre, aunque solo un socket tenga actividad comprueban todos.

select

- **Ficheros de cabecera:**

```
#include <sys/types.h>
```

```
#include <sys/time.h>
```

```
#include <unistd.h>
```

```
#include <sys/socket.h>
```

```
#include <netinet/in.h>
```

```
#include <arpa/inet.h>
```

- Prototipo de **select**:

```
int select(  
    int nfd,  
    fd_set *readfds,  
    fd_set *writefds,  
    fd_set *exceptfds,  
    struct timeval *timeout  
);
```

Devuelve: **> 0** Números de descriptores listos, **0** Timeout agotado y **-1** Error

select

- **fd_set**
 - Es la estructura utilizada para almacenar el conjunto de descripciones a monitorizar.
 - **fd_set readfds;** // es una mascara de bits, cada bit representa un descriptor de fichero.
 - FD 0 -> stdin
 - FD 1 -> stdout
 - FD 2 -> stderr
 - FD 3 -> socket servidor
 - FD 4 -> cliente 1
 - FD 5 -> cliente 2
 - ...

select

- **Macros fundamentales:**

- **FD_ZERO**: inicializar el conjunto → `FD_ZERO(&readfds);`
- **FD_SET**: Añade un descriptor al conjunto → `FD_SET(sockfd, &readfds);`
- **FD_CLR**: Elimina un descriptor → `FD_CLR(client_fd, &readfds);`
- **FD_ISSET**: Comprueba si un descriptor tiene actividad.

```
if (FD_ISSET(client_fd, &readfds)){  
    recv(client_fd, buffer, sizeof(buffer), 0);  
}
```

select

- Parámetros:
 - **select(maxfd + 1, &readfds, NULL, NULL, NULL);**
 - 1) **maxfd + 1** representa el descriptor mas alto + 1 (no el número de sockets)
 - 2) **&readfds**: monitorizar lectura
 - 3) **NULL**: monitorizar escritura (***no interesa***)
 - 4) **NULL**: monitorizar excepciones (***no interesa***)
 - 5) **NULL**: timeout (***espera indefinida***)
 - *La llamada indica al kernel: Tengo varios sockets en readfds. Bloquéame el proceso hasta que alguno tenga datos disponibles para leer."*

select

- **select** monitoriza **3 tipos de eventos**: lectura, escritura y excepciones.
 - **Lectura: readfds**
 - pregunta ¿hay datos disponibles para leer?
 - fd_set readfds
 - **Escritura: writefds**
 - pregunta: ¿puedo escribir sin bloquearme?
 - fd_set writefds;
 - **Excepciones: exceptfds**
 - Detecta condiciones excepcionales.
 - fd_set exceptfds;

select

- Parámetros: **timeout**
- struct timeval {
- long tv_sec;
- long tv_usec;
- };
- #include <sys/time.h>

Ejemplo

- Monitorizar escritura:

```
fd_set writefds;
```

```
FD_ZERO(&writefds);
```

```
FD_SET(sockfd, &writefds);
```

```
select(maxfd + 1, NULL, &writefds, NULL, NULL);
```

Ejemplo

- Monitorizar errores:

```
fd_set exceptfds;
```

```
FD_ZERO(&exceptfds);
```

```
FD_SET(sockfd, &exceptfds);
```

```
select(maxfd + 1, NULL, NULL, &exceptfds, NULL);
```

Ejemplo

- **5 sg de timeout:**

```
struct timeval tv;
```

```
tv.tv_sec = 5;
```

```
tv.tv_usec = 0;
```

```
select(maxfd + 1, &readfds, NULL, NULL, &tv);
```

Ejemplo

- Esperar **indefinida**:
select(..., NULL);
- Consulta inmediata (**polling**)
struct timeval tv;
tv.tv_sec = 0;
tv.tv_usec = 0;
select(..., &tv);

select

- Lo más habitual en un servidor TCP:
select(maxfd + 1,
 &readfds, NULL, NULL, NULL);
- Con esto detectaríamos:
 - si llega una nueva conexión (accept()),
 - si un cliente envía datos (recv()),
 - si un cliente se desconecta.

poll

- **poll()** aparece como evolución de **select()** para eliminar algunas de sus limitaciones, especialmente:
 - el límite de `FD_SETSIZE`,
 - el uso de máscaras de bits (`fd_set`),
 - la gestión poco cómoda de grandes cantidades de sockets.
- El objetivo sigue siendo el mismo:
- Esperar hasta que uno o varios descriptores tengan actividad.

poll

- Ficheros de cabecera: `#include <poll.h>`
- También aparecerán:
 - `#include <sys/socket.h>`
 - `#include <netinet/in.h>`
 - `#include <arpa/inet.h>`
 - `#include <unistd.h>`

poll

- `int poll(struct pollfd *fds, nfds_t nfd, int timeout);`
- Devuelve:
 - > 0 Número de descriptors activos
 - 0 Timeout agotado
 - 1 Error

Parámetro fds de poll

- A diferencia de select(), aquí **no se utilizan conjuntos** (fd_set).
- Cada descriptor se registra dentro de una estructura:
struct pollfd {
 int fd;
 short events;
 short revents;
};

poll: campos struct pollfd

- Campo **fd**
 - Contiene el descriptor a monitorizar
 - `fds[0].fd = server_fd;`
 - `fds[1].fd = client_fd;`
- Campo **events**
 - Indica los eventos que hay vigilar
 - `fds[0].events = POLLIN;`
 - Avísame cuando haya datos disponibles para leer
- Campo **revents**
 - Lo rellena el kernel cuando `poll()` retorna
 - Contendrá los eventos que realmente se han producido

```
if (fds[i].revents & POLLIN){
    recv(...);
}
```

Parámetro nfds de poll

- nfds: Número de elementos del array pollfd.
- struct pollfd fds[100];
- poll(fds, 100, -1);

Parámetro timeout de poll

- El timeout es mas sencillo que en select.
- Espera indefinida: `poll(fds, nfds, -1);`
- Espera inmediata: `poll(fds, nfds, 0);`
- Espera determinada: `poll(fds, nfds, 5000); // 5000 mls`

Eventos poll

- **POLLIN**
 - Datos disponibles para leer:
 - `fds[i].events = POLLIN;`
- **POLLOUT**
 - El descriptor puede escribir sin bloquear
 - `fds[i].events = POLLOUT; // Equivalente al writefds de select().`
- **POLLERR**
 - Error en el descriptor:
 - `if (fds[i].revents & POLLERR){ ... }`

Eventos poll II

- **POLLHUP**

- La conexión ha sido cerrada.
- Muy frecuente en servidores TCP.

```
if (fds[i].revents & POLLHUP){  
    close(fds[i].fd);  
}
```

- **POLLNVAL**

- Descriptor no válido
- close(sockfd) y después se intenta monitorizar.

Flujo típico

- **Configuración:**

```
struct pollfd fds[10];
```

```
fds[0].fd = server_fd;
```

```
fds[0].events = POLLIN;
```

```
fds[1].fd = client_fd;
```

```
fds[1].events = POLLIN;
```

- **Espera:**

```
int ready;
```

```
ready = poll(fds, 2, -1);
```

- **Comprobación:**

```
if (fds[0].revents & POLLIN){
```

```
    // nueva conexión
```

```
}
```

```
if (fds[1].revents & POLLIN){
```

```
    // datos recibidos
```

```
}
```

Resumen

- **poll()** es una evolución de **select()** basada en arrays de estructuras **pollfd** en lugar de máscaras de bits.
- Permite **monitorizar** múltiples sockets **sin** las **limitaciones** de **FD_SETSIZE** y ofrece una **API más sencilla** para gestionar lectura, escritura, errores y desconexiones.
- Aunque resulta más flexible que **select()**, **sigue recorriendo todos los descriptors monitorizados en cada llamada**, por lo que su **rendimiento decrece** cuando el número de conexiones es muy elevado.
- Esta limitación motivó la aparición de **epoll()**, **solución nativa de Linux para servidores de alta concurrencia**.

epoll

- Alto rendimiento en Linux
- `epoll()` es el mecanismo de multiplexación de E/S diseñado específicamente para Linux.
- Su objetivo es resolver el principal problema de: `select` / `poll`
- El problema principal de las dos anteriores es que tienen que recorrer continuamente todos los sockets registrados, aunque la mayoría no tengan actividad.

epoll

- Ficheros de cabecera: `#include <sys/epoll.h>`
- También:
 - `#include <sys/socket.h>`
 - `#include <netinet/in.h>`
 - `#include <arpa/inet.h>`
 - `#include <unistd.h>`

epoll

- Comparativa: select, poll y epoll
- Con **select**:
 - El Kernel tiene que revisar todos los sockets.
- Con **poll**:
 - El Kernel revisa toda la lista de sockets.
- Con **epoll**:
 - El Kernel, registra todos los sockets y avisa solo en caso de que ocurra algo. Notifica por eventos.

epoll

- Flujo: `epoll_create1()` → `epoll_ctl()` → `epoll_wait()`
- **`epoll_create1()`**
 - Crea una instancia de epoll
 - `int epoll_create1(int flags)`
 - **Uso:**
 - `int efpd;`
 - `efpd = epoll_create1(0);`

epoll

- Estructura principal: **struct epoll_event**

- La configuración se realiza con:

```
struct epoll_event {  
    uint32_t events;  
    epoll_data_t data;  
};
```

Campo events

- Los eventos que queremos monitorizar
- `event.events = EPOLLIN;`

Campo data

- Permite asociar información al evento
- `event.data = sockfd`

epoll

- **epoll_ctl()**
 - Permite gestionar los sockets registrados.
 - `int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);`
 - Añadir un socket:
 - `epoll_ctl(epfd, EPOLL_CTL_ADD, sockfd, &event);`
 - Modificar un socket: `EPOLL_CTL_MOD`
 - Eliminar un socket: `EPOLL_CTL_DEL`

epoll

- **epoll_wait()**

- Es la llamada de espera de eventos

- `int epoll_wait(int epfd, struct epoll_event *events, int maxevents, int timeout);`

- **Parámetros:**

- **epfd**: instancia devuelta por `epoll_create1()`

- **events**: Array donde el kernel escribirá los eventos ocurridos:

- `struct epoll_event events[64];`

- **maxevents**: Número máximo de eventos que queremos recibir. 64

- **timeout**: Tiempo de espera en milisegundos

epoll

- **Timeouts**

- Espera **indefinida**:

- `epoll_wait(epfd, events, 64, -1);`

- Espera **inmediata**:

- `epoll_wait(epfd, events, 64, 0); // 0 sg.`

- Espera **limitada**:

- `epoll_wait(epfd, events, 64, 5000); // 5 sg.`

epoll

- **Eventos:**
 - **EPOLLIN**
 - Hay datos disponibles para leer, equivale a POLLIN
 - **EPOLLOUT**
 - El socket está listo para escribir. `event.events = EPOLLOUT;`
 - **EPOLLERR**
 - Ha ocurrido un error
`if(events[i].events & EPOLLERR){ ... }`
 - **EPOLLHUP**
 - La conexión se ha cerrado, habitual para detectar clientes desconectados.

Ejemplo

- **Registrar un socket:**

```
struct epoll_event ev;  
ev.events = EPOLLIN;  
ev.data.fd = server_fd;  
epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);
```

- **Esperar eventos:**

```
struct epoll_event events[64];  
int n;  
n = epoll_wait(epfd, events, 64, -1);
```

- **Procesar los eventos:**

```
for(int i = 0; i < n; i++){  
    if(events[i].events & EPOLLIN) {  
        printf("Actividad en fd=%d\n", events[i].data.fd);  
    }  
}
```

Resumen

- **epoll()** es el mecanismo de multiplexación de E/S de **alto rendimiento** de Linux.
- A diferencia de **select()** y **poll()**, mantiene los **descriptores registrados** dentro del kernel y **devuelve** únicamente **aquellos que presentan actividad**.
- Se basa en las funciones `epoll_create1()`, `epoll_ctl()` y `epoll_wait()`, junto con la estructura `epoll_event` para registrar y recibir eventos.
- Gracias a su modelo orientado a eventos, permite construir servidores concurrentes capaces de manejar decenas o cientos de miles de conexiones con un uso reducido de CPU y memoria.

Comparativa select / poll / epoll

Característica	select()	poll()	epoll()
Portable Unix	✓	✓	✗
Fácil de aprender	✓	✓	⚠
Límite FD_SETSIZE	✓ Problema	✗	✗
Recorre todos los sockets	✓	✓	✗
Escala a miles de conexiones	✗	⚠	✓
Alto rendimiento Linux	✗	⚠	✓

Servidores Concurrentes

- Modelo por hilos – Un hilo por cliente
- Modelo por procesos – **fork()**
- Modelo reactivo – **select / epoll**

Modelo de hilos

- **Es el mas intuitivo**

Llega una conexión

|

v

accept()

|

v

pthread_create()

|

v

El hilo atiende
únicamente a ese
cliente

- **Funcionamiento:**

- El hilo principal se encarga únicamente de:

- crear el socket servidor,
- ejecutar listen(),
- aceptar conexiones mediante accept().

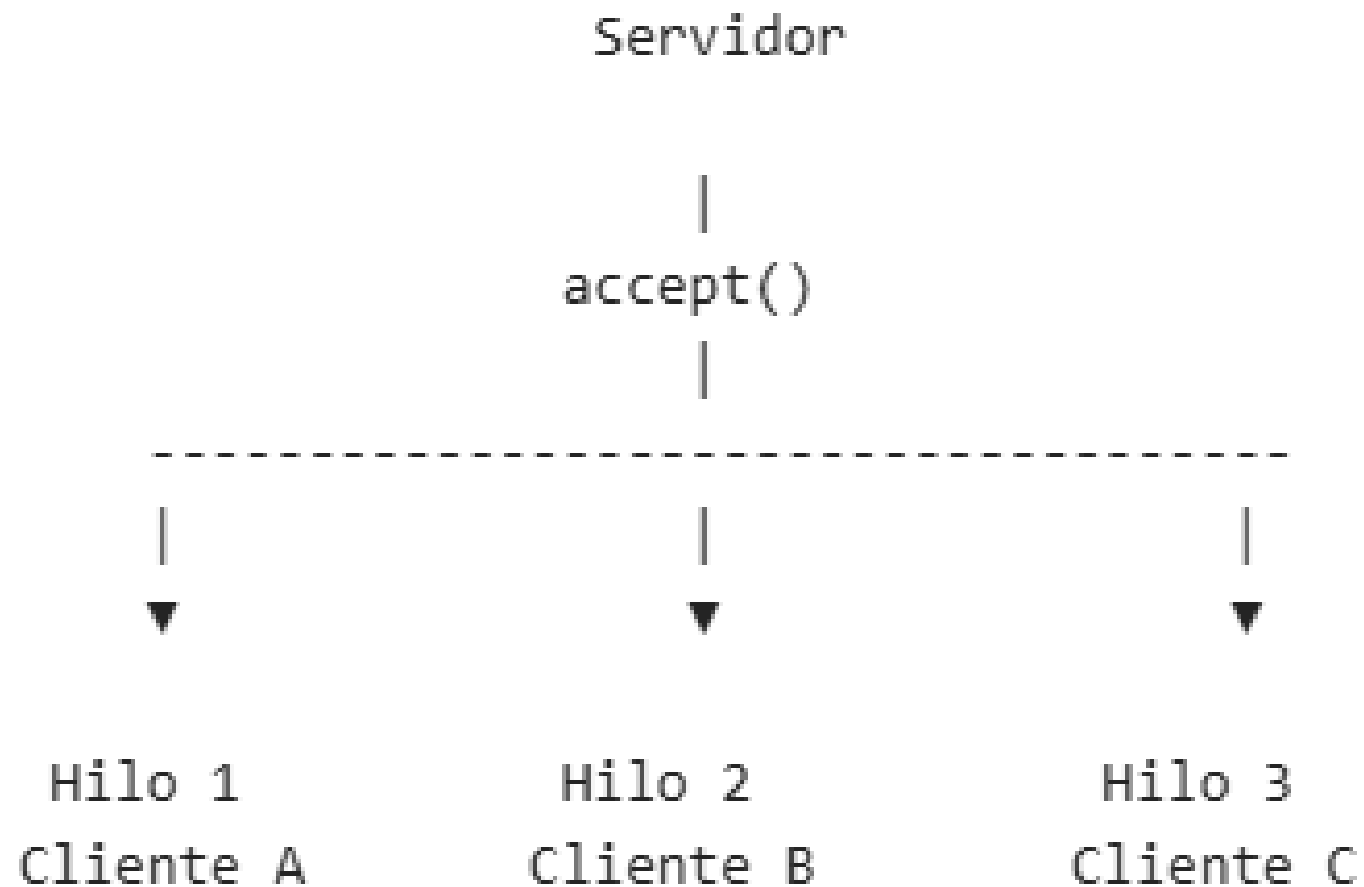
- Cada que llega un cliente:

- client_fd = accept(...);

- Se crear el nuevo hilo:

- pthread_create(. . .); // se dedica a solo este cliente

Modelo de hilos



Modelo de hilos

- Ficheros de cabecera (trabajamos con POSIX y sockets)

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#include <sys/socket.h>
```

```
#include <netinet/in.h>
```

```
#include <arpa/inet.h>
```

Modelo de hilos

- Funciones principales:

- Crear el hilo:

- ```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
void *(*start_routine)(void *), void *arg);
```

- Esperar a que finalice un hilo:

- ```
pthread_join(tid, NULL);
```

- Marcar un hilo como independiente:

- ```
pthread_detach(tid)
```

```
while (1)
{
 client_fd = accept(server_fd, NULL, NULL);
 if (client_fd < 0) {
 perror("accept");
 continue;
 }
 pthread_t tid;
 pthread_create(&tid, NULL, client_handler, &client_fd);
 pthread_detach(tid);
}
```

```
void *client_handler(void *arg){
 int client_fd = *(int *)arg;
 // Atender al cliente ...
 close(client_fd);
 return NULL;
}
```

# Problema del ejemplo

- **&client\_fd**
  - Apunta a una variable que se reutiliza en cada iteración del bucle.
  - Si llegan varias conexiones muy rápido, varios hilos podrían acabar leyendo el mismo descriptor.
  - Por eso, en una implementación real normalmente se:
    - reserva memoria para cada cliente,
    - o se utiliza una estructura específica por conexión.

```
while (1){
 int client_fd = accept(server_fd, NULL, NULL);
 if (client_fd < 0) {
 perror("accept");
 continue;
 }
 int *fd = malloc(sizeof(*fd));
 if (fd == NULL) {
 close(client_fd);
 continue;
 }
 *fd = client_fd;
 pthread_t tid;
 pthread_create(&tid, NULL, client_handler, fd);
 pthread_detach(tid);
}
```

```
void *client_handler(void *arg){
 int client_fd = *(int *)arg;
 free(arg);
 /* atender cliente */
 close(client_fd);
 return NULL;
}
```

# Modelo de procesos

- Antes de la popularización de los hilos POSIX, muchos servidores Unix utilizaban un modelo basado en procesos.
- La idea es muy similar al modelo por hilos:

Llega una conexión

|

accept()

|

fork()

|

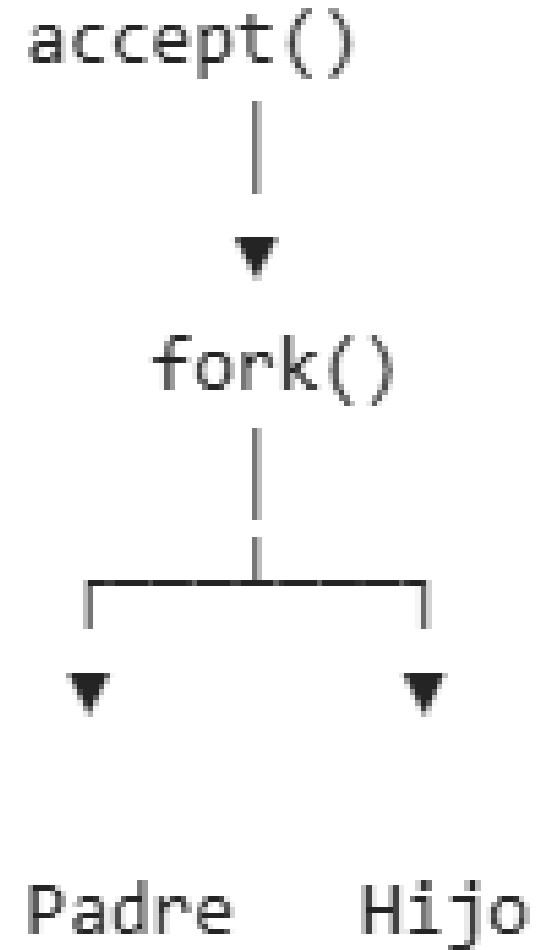
Proceso hijo atiende  
exclusivamente al cliente

# Modelo de procesos

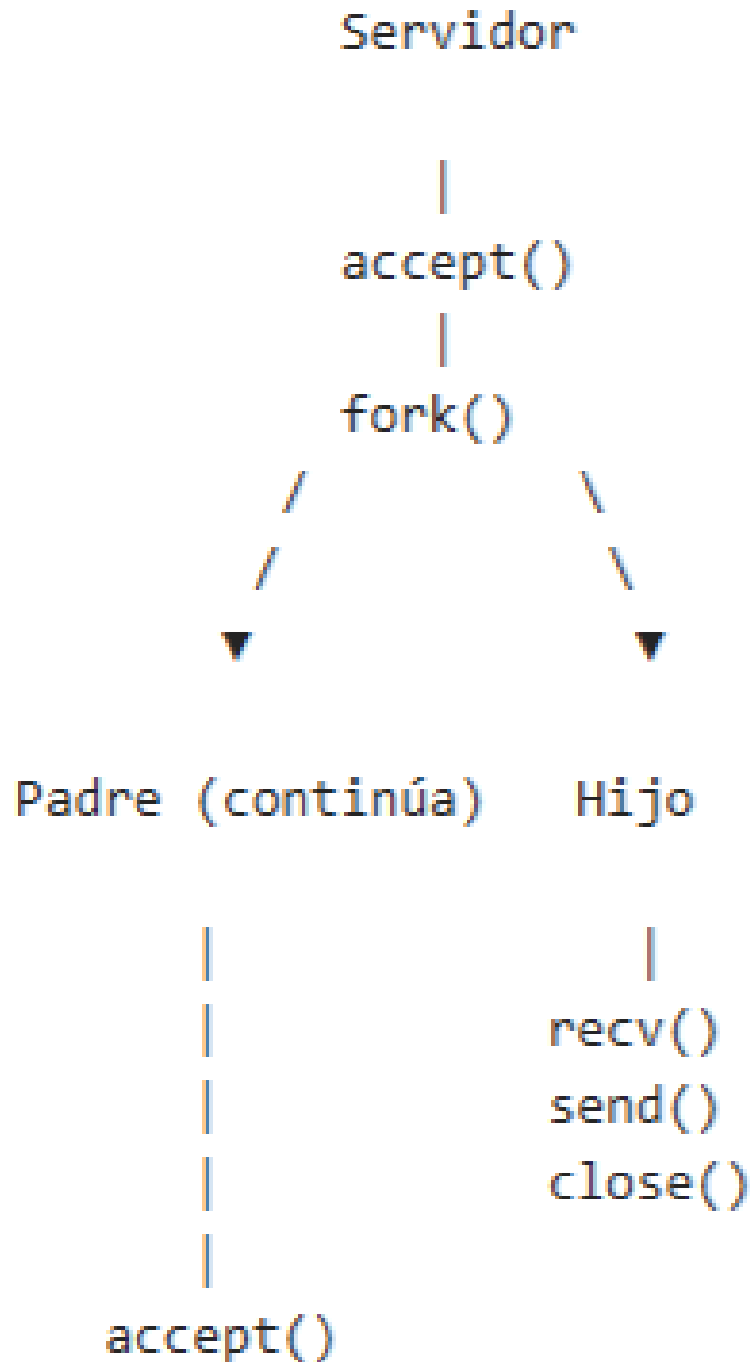
- **fork()**
  - Es una llamada al sistema que crea una copia del proceso actual.
  - `#include <unistd.h>`
  - `pid_t fork(void);`
  - El proceso **padre** recibe el PID del hijo **pid > 0**
  - Y el proceso **hijo** recibe un 0, **pid == 0**

# Modelo de procesos

- El padre continúa aceptando conexiones.
- El hijo atiende exclusivamente al cliente.



# Modelo de procesos



```
while (1){
 int client_fd;
 client_fd = accept(server_fd, NULL, NULL);

 if (client_fd < 0)
 continue;
 pid_t pid = fork();

 if (pid == 0) {
 /* Proceso hijo */
 close(server_fd);
 /* atender cliente */
 close(client_fd);
 exit(EXIT_SUCCESS);
 }
 /* Proceso padre */
 close(client_fd);
}
```

# Modelo de procesos

- Aislamiento completo:
  - Cada cliente se ejecuta en un proceso independiente.
- Espacio de memoria separado:
  - Cada proceso posee su propia memoria.
  - No existen problemas como:
    - race conditions,
    - acceso simultáneo a variables,
    - corrupción accidental de datos compartidos.

# Modelo de procesos

- Creación costosa:
  - Crear un proceso es mucho mas caro que crear un hilo
  - **fork()** consumen mas recursos que **pthread\_create()**
- Mayor consumo de memoria:
  - Cada proceso mantiene:
    - tablas de memoria,
    - contexto de ejecución,
    - recursos propios.

Problema de procesos **Zombie**  
Cuando el hijo finaliza: `exit(EXIT_SUCCESS)`  
  
Su entrada continua, existiendo temporalmente  
En la tabla de procesos.

# Modelo de procesos

- Utilizar este modelo con:
  - servidores Unix clásicos,
  - demonios del sistema,
  - aplicaciones donde prima el aislamiento,
  - herramientas administrativas.

# Comparativa procesos / hilos

| Característica      | Hilos      | Procesos     |
|---------------------|------------|--------------|
| Creación            | Rápida     | Más costosa  |
| Memoria             | Compartida | Separada     |
| Aislamiento         | Bajo       | Alto         |
| Comunicación        | Fácil      | Más compleja |
| Consumo de recursos | Menor      | Mayor        |
| Escalabilidad       | Mejor      | Peor         |

# Resumen

- El modelo por procesos utiliza **fork()** para crear un proceso hijo por cada conexión aceptada.
- El proceso padre continúa ejecutando **accept()** mientras que cada proceso hijo atiende de forma exclusiva a un cliente concreto.
- Este modelo proporciona un alto aislamiento entre conexiones y evita muchos problemas asociados a la memoria compartida, aunque presenta un mayor consumo de recursos y un coste de creación superior al modelo basado en hilos.
- Históricamente ha sido muy utilizado en servidores Unix clásicos y sigue siendo una opción válida cuando se prioriza robustez y aislamiento frente a escalabilidad masiva.

# Ejemplo, evitar procesos zombies

```
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
while (1)
{
 /* Recoger los procesos hijo finalizados sin bloquear */
 while (waitpid(-1, NULL, WNOHANG) > 0) {
 /* No es necesario procesar su estado */
 }
 int client_fd = accept(server_fd, NULL, NULL);
 if (client_fd < 0) {
 continue;
 }
}
```

```
pid_t pid = fork();
if (pid < 0) {
 /* Error al crear el proceso hijo */
 close(client_fd);
 continue;
}
if (pid == 0) {
 /* Proceso hijo */
 close(server_fd);
 /* Atender al cliente */
 close(client_fd);
 exit(EXIT_SUCCESS);
}
/* Proceso padre */
close(client_fd);
}
```

# Ejecución de comandos

- Desde un programa C, podemos ejecutar comandos del sistema operativo.
- `#include <unistd.h>`
- `int system(const char *cmdstring);`
  - **system()** crea un proceso que ejecuta un shell y le pasa el comando para que lo ejecute.
- Devuelve el código retornado por el comando de shell, 127 si no pudo ejecutar el shell y -1 en caso de otro error.

# Creación de procesos

- La llamada al sistema **fork()** crea un proceso:
- `#include <unistd.h>`
- `pid_t fork(void);`
- Se crea un proceso **idéntico al padre**.
- `fork()` devuelve 0 al proceso hijo, el PID del proceso hijo al padre, -1 en caso de error.
- Cada proceso queda identificado en la máquina por un número, llamado pid (process id).

# Funciones de terminación

- **\_exit()** vuelve al kernel inmediatamente. Definida por POSIX:
  - #include <unistd.h>
  - void \_exit(int status);
- **exit()** realiza antes cierto “limpiado” (e.g. terminar de escribir los buffers a disco). Es ANSI C:
  - #include <stdlib.h>
  - void exit(int status);
- **abort()** *el proceso se envía a sí mismo la señal SIGABRT. Termina y produce un **core dump** (volcado de memoria).*

# Espera por el proceso hijo

- `#include <sys/types.h>`
- `#include <sys/wait.h>`
- `pid_t wait(int *statloc);`
- `pid_t waitpid(pid_t pid, int *statloc, int options);`
- Suspende al proceso que la ejecuta hasta que alguno de sus hijos termina.
- Si algún hijo ha terminado se devuelve el resultado inmediatamente.
- El valor retornado por el proceso hijo puede deducirse del valor devuelto en `statloc`.

# Ejemplo

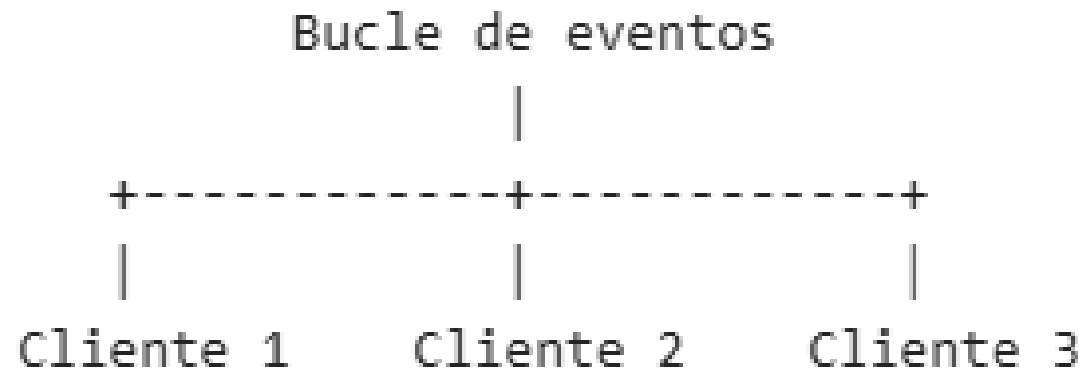
```
if (fork()==0){
 printf ("Proceso hijo!!! \n");
 exit(1);
} else { // Proceso padre espera a que termine el hijo
 int mark;
 wait(&mark);
}
```

# Ejecutar otro programa

- **exec()** se emplea para lanzar un programa de un fichero.
- El programa en ejecución en el proceso llamante es sustituido por el programa que se ejecuta, el PID no cambia.
- `char* argumentos [] = { "ls", "-l", "/usr/include", 0 };`
- `execv ( "/bin/ls", argumentos );`
- Las llamadas retornan un valor no nulo si no se puede ejecutar el programa.

# Modelo Reactivo

- **select / epoll**
  - El modelo reactivo utiliza **un único hilo o proceso** para gestionar múltiples clientes.
  - En lugar de asignar un hilo o proceso a cada conexión, el servidor espera eventos de entrada/salida y procesa únicamente los sockets preparados.



# Modelo reactivo con select()

El servidor mantiene:

- El socket de escucha.
- Los sockets de los clientes.
- Un conjunto maestro de descriptores.
- Una copia del conjunto para cada llamada a select().
- select() modifica el conjunto recibido, dejando únicamente los descriptores preparados.
  - Solo admite descriptores inferiores a FD\_SETSIZE, normalmente 1024 en Linux.

# Ejemplo

```
fd_set master_set;
fd_set ready_set;
FD_ZERO(&master_set);
FD_SET(server_fd, &master_set);
int max_fd = server_fd;
while (1) {
 ready_set = master_set;
 int ready = select(max_fd + 1, &ready_set,
 NULL, NULL, NULL);
 if (ready < 0) {
 perror("select");
 break;
 }
 for (int fd = 0; fd <= max_fd; ++fd) {
 if (!FD_ISSET(fd, &ready_set)) {
 continue;
 }
 }
}
```

```
if (fd == server_fd) {
 int client_fd = accept(server_fd, NULL, NULL);
 if (client_fd >= 0) {
 FD_SET(client_fd, &master_set);
 if (client_fd > max_fd) {
 max_fd = client_fd;
 }
 }
 else {
 char buffer[1024];
 ssize_t received = recv(fd, buffer, sizeof(buffer), 0);
 if (received <= 0) {
 close(fd);
 FD_CLR(fd, &master_set);
 } else {
 send(fd, buffer, (size_t)received, 0);
 }
 }
}
}
```

# Funcionamiento

- **server\_fd** está preparado para lectura cuando hay una conexión pendiente.
- Los sockets cliente están preparados cuando hay datos o se produce un cierre.
- **accept()** registra nuevos clientes.
- **recv()** procesa los clientes activos.
- **FD\_CLR()** elimina conexiones cerradas.

# Modelo reactivo con epoll()

- epoll mantiene en el kernel una **lista de interés** con los descriptors registrados y una **lista de preparación** con aquellos que tienen eventos pendientes.
- La aplicación **registra sockets** mediante **epoll\_ctl()** y obtiene los **activos** mediante **epoll\_wait()**.

# Ejemplo: creación y registro del servidor

```
int epoll_fd = epoll_create1(0);
if (epoll_fd < 0) {
 perror("epoll_create1");
 return 1;
}
// Configurar el tipo de evento
struct epoll_event event;
event.events = EPOLLIN;
event.data.fd = server_fd;

if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, server_fd, &event) < 0) {
 perror("epoll_ctl");
 return 1;
}
```

# Ejemplo: Bucle principal simplificado

Este ejemplo emplea el modo **level-triggered**, que es el funcionamiento predeterminado cuando no se especifica **EPOLLET**.

```
#define MAX_EVENTS 64
struct epoll_event events[MAX_EVENTS];

while (1) {
 int count = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);
 if (count < 0) {
 perror("epoll_wait");
 break;
 }

 for (int i = 0; i < count; ++i) {
 int fd = events[i].data.fd;
 if (fd == server_fd) {
 int client_fd = accept(server_fd, NULL, NULL);
 if (client_fd < 0) {
 continue;
 }
 }
 }
}
```

```
struct epoll_event client_event;
 client_event.events = EPOLLIN;
 client_event.data.fd = client_fd;

 if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, client_fd, &client_event) < 0) {
 close(client_fd);
 }

} else {
 char buffer[1024];
 ssize_t received = recv(fd, buffer, sizeof(buffer), 0);
 if (received <= 0) {
 epoll_ctl(epoll_fd, EPOLL_CTL_DEL, fd, NULL);
 close(fd);
 } else {
 send(fd, buffer, (size_t)received, 0);
 }
}
}
}
```

# Diferencias select() vs epoll()

- **select()**

- Reconstruir conjunto
- Esperar
- Recorrer descriptores
- Procesar activos

- No se crea un hilo ni proceso por cliente.
- Un bucle central procesa eventos de todos los sockets.
- select() es apropiado para aprendizaje y pocos descriptores.
- epoll() está diseñado para gestionar grandes cantidades de descriptores en Linux.
- El código debe evitar operaciones lentas o bloqueantes dentro del bucle de eventos.

- **epoll()**

- Registrar descriptores una vez
- Esperar
- Recibir únicamente eventos preparados
- Procesar activos

# Protocolos y Serialización

- Protocolos binarios – endianness, alineamiento
- Serialización – estructuras empaquetadas

# Protocolos binarios – endianness, alineamiento

- Partimos de una estructura de C y hay que transformarla en una secuencia de bytes para enviarse por TCP / UDP y reconstruir correctamente en el receptor.

# Protocolos Binarios

Un protocolo binario define con precisión:

- Tipos de mensajes;
- Orden y tamaño de los campos;
- Orden de los bytes;
- Longitud del mensaje;
- Tratamiento de errores;
- Versión del protocolo.

# Ejemplo de formatos

- Byte 0                      Versión                      1 byte
  - Byte 1                      Tipo                      1 byte
  - Bytes 2-3                      Longitud                      2 bytes
  - Bytes 4-7                      Identificador                      4 bytes
  - Bytes 8...                      Datos                      longitud variable
- 
- Los tipos de **tamaño fijo** de **<stdint.h>** evitan depender del tamaño de int o long.  

```
#include <stdint.h>
uint8_t version;
uint16_t length;
uint32_t identifier;
```

# Endianness

- Endianness determina el orden en que se almacenan los bytes de un número formado por varios bytes.
- `uint32_t value = 0x12345678;`
- |    |    |    |    |
|----|----|----|----|
| b3 | b2 | b1 | b0 |
|----|----|----|----|
- **Little Endian**
  - El byte menos significativo se almacena el primero. *Es el orden habitual en arquitecturas x86 y x86-64.*
    - Dirección baja                      Dirección alta
    - 78                      56                      34                      12
- **Big Endian**
  - El byte más significativo se almacena el primero.
    - Dirección baja                      Dirección alta
    - 12                      34                      56                      78
  - *El orden de red utilizado por los protocolos de Internet es big-endian. Linux proporciona funciones para convertir enteros de 16 y 32 bits entre el orden del equipo y el orden de red.*

# Funciones de conversión

- Fichero de cabecera: `#include <arpa/inet.h>`
- **Del Host a la red:**
  - `uint16_t htons(uint16_t value);`
    - `htons`: host to network short, 16 bits.
  - `uint32_t htonl(uint32_t value);`
    - `htonl`: host to network long, 32 bits.
- **De la red al Host:**
  - `uint16_t ntohs(uint16_t value);`
    - `ntohs`: network to host short.
  - `uint32_t ntohl(uint32_t value);`
    - `ntohl`: network to host long

# Ejemplo

- **En el emisor:**

```
uint16_t port = 8080;
uint16_t network_port = htons(port);
uint32_t identifier = 1000;
uint32_t network_identifier = htonl(identifier);
```

- **En el receptor:**

```
port = ntohs(network_port);
identifier = ntohl(network_identifier);
```

# Alineamiento y padding

- El compilador puede introducir bytes de relleno dentro de una estructura:

```
typedef struct {
 uint8_t type; // 1 byte
 uint32_t identifier; // 4 bytes
} message_t;
```

- La estructura puede ocupar 8 bytes:
  - Offset 0            type
  - **Offsets 1-3      padding**
  - Offsets 4-7      identifier

# Ejemplo

```
#include <stddef.h>
```

```
#include <stdio.h>
```

```
printf("Tamaño: %zu\n", sizeof(message_t));
```

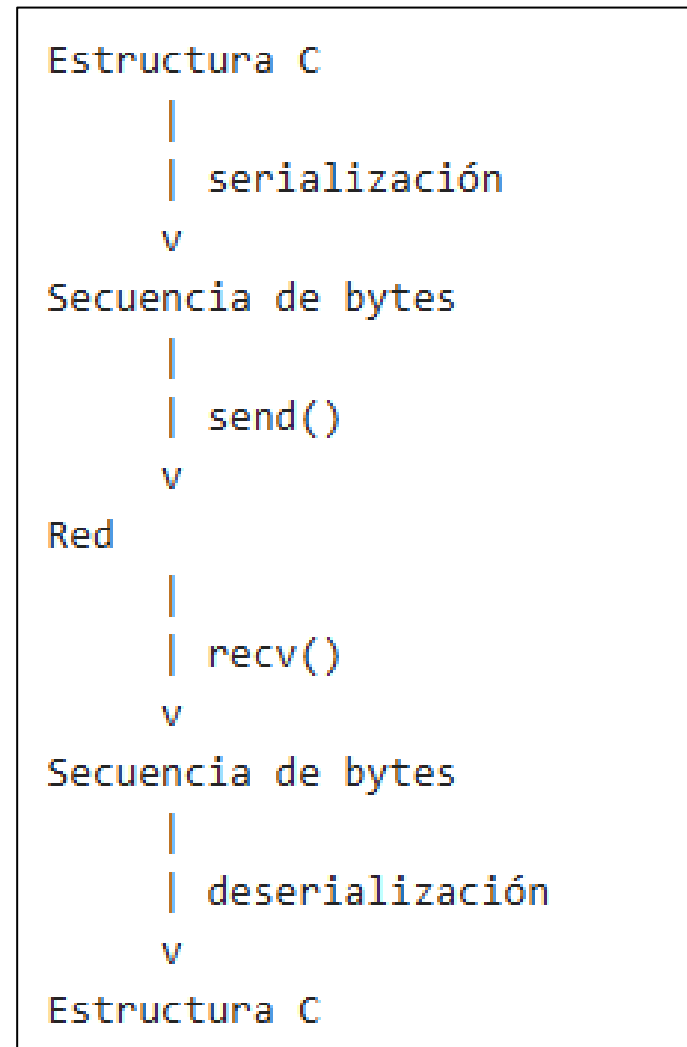
```
printf("Offset type: %zu\n", offsetof(message_t, type));
```

```
printf("Offset identifier: %zu\n", offsetof(message_t, identifier));
```

// Enviar directamente una estructura normal puede transmitir también el padding, cuya disposición depende del compilador y de la plataforma.

# Serialización – estructuras empaquetadas

- Serializar consiste en convertir datos internos a un formato de bytes definido por el protocolo.
- Deserializar, la operación inversa.



# Serialización

```
#include <arpa/inet.h>
#include <stdint.h>
#include <string.h>
```

```
#define HEADER_SIZE 8
```

```
typedef struct {
 uint8_t version;
 uint8_t type;
 uint16_t length;
 uint32_t identifier;
} message_header_t;
```

```
void serialize_header(uint8_t buffer[HEADER_SIZE], const message_header_t *header) {
 uint16_t length = htons(header->length);
 uint32_t identifier = htonl(header->identifier);
 buffer[0] = header->version;
 buffer[1] = header->type;
 memcpy(buffer + 2, &length, sizeof(length));
 memcpy(buffer + 4, &identifier, sizeof(identifier));
}
```

**El buffer tiene esta distribución:**

|     |                            |
|-----|----------------------------|
| 0   | versión                    |
| 1   | type                       |
| 2-3 | length en orden de red     |
| 4-7 | identifier en orden de red |

# Deserialización

```
void deserialize_header(message_header_t *header, const uint8_t buffer[HEAD*R_SIZE])
{
 uint16_t length;
 uint32_t identifier;

 header->version = buffer[0];
 header->type = buffer[1];

 memcpy(&length, buffer + 2, sizeof(length));
 memcpy(&identifier, buffer + 4, sizeof(identifier));

 header->length = ntohs(length);
 header->identifier = ntohl(identifier);
}
```

# Estructuras empaquetadas

- Se puede eliminar el padding:

```
#pragma pack(push, 1)
```

```
typedef struct {
 uint8_t version;
 uint8_t type;
 uint16_t length;
 uint32_t identifier;
} packed_header_t;
#pragma pack(pop)
```

`sizeof(packed_header_t)`

Ocupará 8 bytes

# Estructuras empaquetadas

- Una estructura empaquetada puede eliminar el padding, pero no resuelve automáticamente el endianness.

```
packed_header_t header= {
 .version = 1,
 .type = 2,
 .length = htons(128),
 .identifier = htonl(1000)
};
```

Acceder directamente a sus campos puede provocar accesos desalineados.

Para protocolos portables se recomienda la **serialización** manual con **buffers** y **memcpy()**.

# TCP y límites de los mensajes

- TCP transporta un flujo de bytes no mensajes independientes.
- Una llamada: **send**(fd, buffer, 8, 0);
- No garantiza que el receptor obtenga los ocho bytes en un único: **recv**(), puede recibir: 3 bytes + 5 bytes.
- Por ello, el protocolo debe incluir una longitud y la aplicación debe acumular datos hasta completar el mensaje:

# TCP y límites de los mensajes

## Cabecera fija

+-- longitud del cuerpo

|

+-- cuerpo variable

## Cabecera fija, 8 bytes

└── Versión: 1 byte

└── Tipo: 1 byte

└── Longitud: 2 bytes

└── Identificador: 4 bytes

## Cuerpo variable

└── Número de bytes indicado por Longitud

# Resumen

- Un protocolo binario debe definir tamaños, offsets, endianness y longitudes.
- Deben utilizarse tipos como `uint8_t`, `uint16_t` y `uint32_t`.
- Los enteros multibyte se convierten mediante `htons()`, `htonl()`, `ntohs()` y `ntohl()`.
- El padding impide asumir que una estructura ocupa la suma de sus campos.
- `#pragma pack` elimina padding, pero no corrige endianness ni garantiza accesos alineados.
- La solución más portable es serializar manualmente en un buffer mediante `memcpy()`.
- En TCP se deben gestionar lecturas y escrituras parciales.